by KHUSHI MEHTA

Made with GAMMA

Perceptron vs Logistic Regression on a Synthetic Dataset

Course Name: Deep Learning

Student Name: Khushi Mehta

Roll Number: 102303769

Subgroup: 3C54

Made with GAMMA

Dataset Definition and Construction

Dataset Representation:

- Two input features: x_1 (Theory Score), x_2 (Practical Score)
- Binary classification: Class 0 (Fail), Class 1 (Pass)

Dataset Parameters (Roll Number: 102303769):

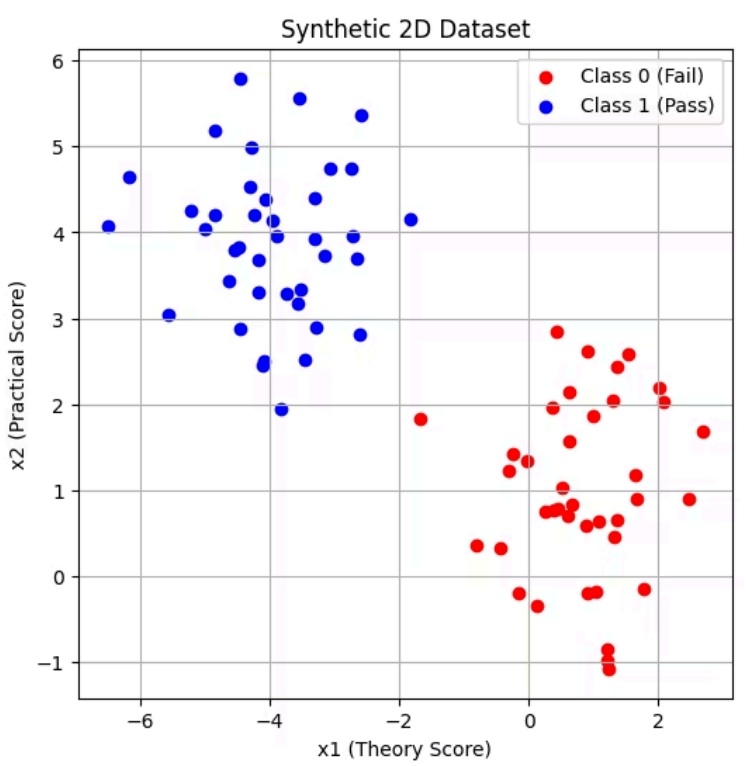
- Last digit a = 9, Second-last digit b = 6
- Points per class: 30 + a = 39 points per class (78 total samples)
- Class 1 rotation: 15 × b = 90 degrees (orthogonal rotation)
- Gaussian noise: σ = 0.01 × a = 0.09

Dataset Non-Triviality:

- Rotation produces a non-axis-aligned class distribution
- Added noise introduces overlap, making the data not perfectly linearly separable

Visualization:

- 2D scatter plot with Class 0 and Class 1 points clearly marked
- Decision boundaries are shown in later slides



Made with GAMMA

Perceptron Model and Decision Boundary

Perceptron Decision Equation:

ŷ = $sign(w_1x_1 + w_2x_2 + b)$

- Hard-threshold classifier with no probabilistic output

Learning Configuration:

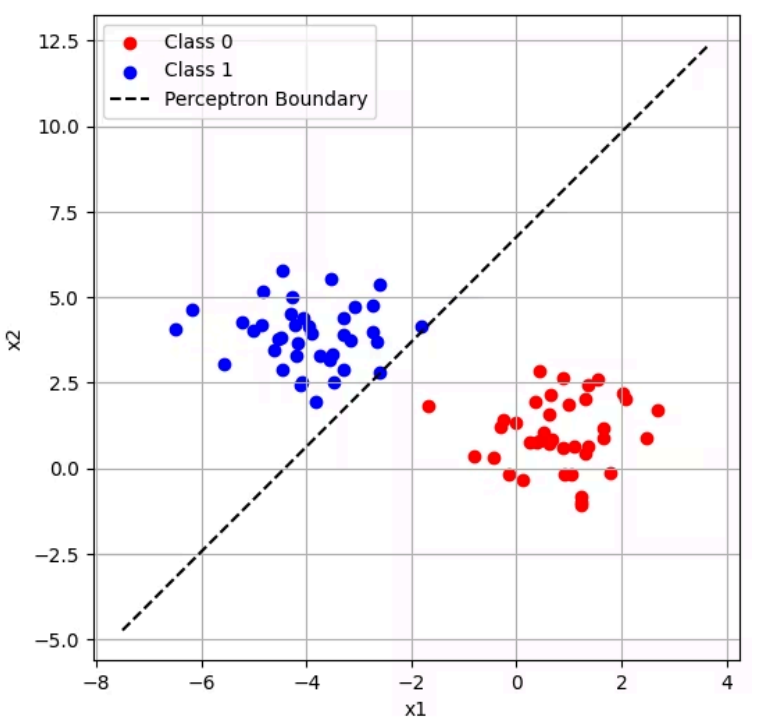
- Learning rate η = 0.01
- Training epochs: 100 (sufficient for convergence on this dataset size)

Observed Behavior:

- A linear decision boundary is learned on the dataset
- Due to 90° rotation, the boundary is not axis-aligned
- Some misclassifications occur in overlapping noisy regions

Visualization:

- Linear decision boundary overlaid on the 2D scatter plot
- Boundary orientation reflects the rotated class structure



Made with GAMMA

Perceptron Behavior: Convergence and Learning Rate Effects

Observed Misclassifications

- Points in the overlapping noise region ($\sigma = 0.09$) are frequently misclassified
- These misclassifications are unavoidable due to the non-linear separability introduced by rotation and noise

Convergence Analysis

- With $\eta = 0.01$, the perceptron converges within 100 epochs
- Convergence is stable; no oscillation observed
- The algorithm finds a separating hyperplane that minimizes training errors

Learning Rate Sensitivity

- If η is increased (e.g., $\eta = 0.1$): Larger weight updates lead to faster convergence but risk overshooting the optimal boundary, causing oscillation
- If η is decreased (e.g., $\eta = 0.001$): Slower convergence but more stable trajectory; may require more epochs to reach the boundary
- Optimal $\eta = 0.01$ balances speed and stability for this dataset

Relationship to Dataset Properties

- The 90-degree rotation means the decision boundary is not aligned with feature axes
- Gaussian noise creates ambiguous regions where both classes overlap
- The perceptron's hard-threshold nature cannot assign confidence to borderline cases

Made with GAMMA

Logistic Regression: Probabilistic Classification

Logistic Regression Probability Equation:

$$P(y = 1|x) = \sigma(w_1x_1 + w_2x_2 + b) = 1/(1 + e^{-(w_1x_1+w_2x_2+b)})$$

- Output: Probability score in [0, 1]
- Decision threshold: P(y = 1 | x) ≥ 0.5 for Class 1, otherwise Class 0

Learning Configuration:

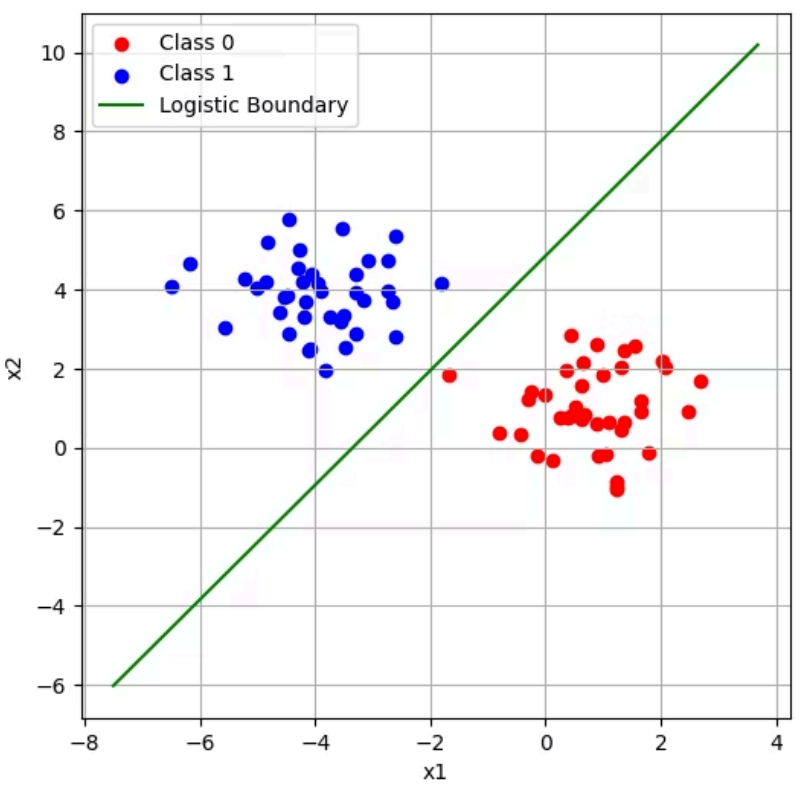
- Learning rate: 0.01 (same as perceptron for fair comparison)
  - Epochs: 100 (same as perceptron for fair comparison)

Decision Boundary on Dataset:

- Linear boundary similar in orientation to the perceptron
- However, logistic regression assigns confidence scores to each prediction

Key Behavioral Difference from Perceptron:

- Perceptron: Hard decision (Class 0 or Class 1)
- Logistic Regression: Soft decision with probability estimates
- For borderline points in the noise region, logistic regression outputs probabilities close to 0.5, reflecting uncertainty



Made with GAMMA

Perceptron vs Logistic Regression: Comparative Insights

Case 1: Where Perceptron Performs Better

- **Scenario:** When the dataset is perfectly linearly separable (no noise)
- **Reason:** The perceptron converges to a separating hyperplane with guaranteed zero training error
- **On our dataset:** Not applicable due to noise and rotation, but theoretically relevant

Case 2: Where Logistic Regression Performs Better

- **Scenario:** When handling noisy, non-separable data (our dataset)
- **Reason:** Logistic regression assigns confidence scores, allowing practitioners to identify uncertain predictions
- **Advantage:** Probability outputs enable threshold adjustment; can trade off precision and recall
- **On our dataset:** Logistic regression provides $P(y=1|x) \approx 0.5$ for borderline points, signaling ambiguity

Technical Insight from This Experiment:

- Linear classifiers (both perceptron and logistic regression) learn the same decision boundary geometry
- The fundamental difference lies in how they handle uncertainty: hard thresholding vs. probabilistic scoring
- For non-linearly separable data, the choice between perceptron and logistic regression depends on whether confidence estimates are needed
- Rotation and noise in the dataset demonstrate that real-world classification problems require models that can express uncertainty

Made with GAMMA